AEGIS USER GUIDE

A guide for the engineering team

28 October 2026 · Companion to AEGIS_TECHNICAL_DOCUMENTATION

AEGIS is a software protection platform. We are building it to make software harder to understand, copy, modify, or reconstruct through reverse engineering, and to measure how much the protection helps.

Our intended scope includes desktop software, websites and web applications, mobile apps, games, server software, embedded systems, and authorized console builds. Different environments need different protection methods. The platform brings those methods into one workflow.

This guide explains the project, your assignment, and how to begin. The Technical Documentation expands the same design into interfaces, build instructions, experiments, and detailed implementation briefs. Read this guide first, then the sections linked from your assignment.

What the developer should be able to do

A developer chooses the part of their software they want to protect and the costs they can accept. AEGIS applies a compatible method, checks that the software still works, and records the result. The planned Adversarial Lab then attempts to reconstruct the protected logic under controlled conditions.

For example, a game studio could protect selected gameplay routines, compare frame time before and after protection, and test whether an analysis agent can recover the same behavior. A web team would need a browser-compatible method and browser tests. The goal is the same; the build path is different.

The parts we are building

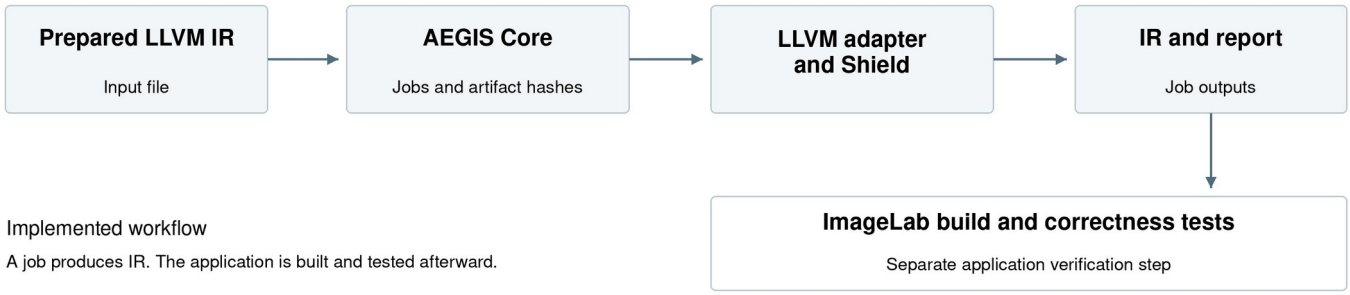
Part	Its purpose
Shield	Transform selected program logic while preserving its intended behavior.
Mirage	Add harmless decoys and test whether they make analysis less effective.
Signature	Sign build records and investigate recoverable software watermarks.
Sentinel	Attach machine-readable rights and usage policies for cooperating tools.
Adversarial Lab	Run authorized reconstruction attempts and score the results.
Platform	Connect the tools, keep artifacts and reports, and later compare protection options.

The next proof we need: a fair comparison showing whether a protected build is harder to reconstruct than its unprotected counterpart, with correctness and performance measured alongside the result.

Deeper reading: Technical Documentation 01 and 02.

What works today

The working foundation is a local command-line tool, an HTTP API, a job runner, and an LLVM protection backend. LLVM is a compiler toolkit; its intermediate representation, or IR, is the form of code our first backend accepts.



The present adapter takes a textual LLVM IR file, applies Shield, and returns transformed IR plus a report. Building the final application and checking its behavior are separate steps today. The intended product will bring these steps together.

The evidence we have

Recorded result	What it tells us
199 Python tests passed	The recorded combined run covered Core, API, CLI, and the real LLVM adapter.
43 arithmetic sites across 3 ImageLab functions	The recorded application job applied the current Shield transformation.
11 of 11 ImageLab tests passed with external IR	The protected output could be rebuilt and passed that configuration's correctness checks.

These are earlier recorded results, not new tests run for this document. The code snapshot is **c995857**. The current transformation has no established resistance to reverse engineering, and later compiler optimization may simplify it away.

Shield v2, Mirage, Signature, Sentinel, the real Adversarial Lab, the optimizer, and the dashboard remain work to build. Core already has authorization and result contracts for Lab jobs; that does not mean a real attack backend exists.

Why we use more than one test application

ImageLab is our small C++ test application. It makes defects easy to isolate and has independent checks for expected output. Keep it as a regression tool.

PhotoCraft, reVC, and Mattermost are the requested external targets: a Rust desktop editor, a native game, and a Go server with a web client. They test different integration problems. None is currently demonstrated as protected by AEGIS.

The engine must work from supported code structures and formats, not application names or memorized answers. Reserve additional, unseen programs for final evaluation. Supporting three examples will not establish support for all software.

Deeper reading: Technical Documentation 05 through 08 and the evidence register in 06.

Four roles, one product

These are technical responsibilities, not person assignments. The same roles appear in Technical Documentation section 10 and GitHub issues 9-12. Each role has one working branch; all changes return to dev/integration through pull requests.

A Core Platform, Shield & Integration

Build the protection engine and the workflow around it. Own Core, API, CLI, LLVM Shield and its adapter, ImageLab, CI and repository integration. Make the existing application build reproducible, then measure which transformations survive optimization and add one tested defense.

First delivery: clean-clone checks, a repeatable protection-to-application run, and a Shield improvement with correctness, final-binary retention, runtime and size evidence. Adaptive optimization follows trustworthy measurements.

Done when: another engineer can reproduce the run; supported and unsupported inputs are handled correctly; failures are visible; protection claims stay within the evidence.

Branch: feat/core-shield. Issue #9. Technical Documentation 10A (two pages).

B Adversarial AI Lab & Mirage

Build the experiment that tests whether protection helps. Run paired Ghidra and model reconstruction attempts with equal access and budgets. Compile candidates in isolation and score them against hidden tests. Then use the same Lab to evaluate harmless decoys.

First delivery: one reproducible baseline/protected comparison with hashes, model settings, budgets, transcripts and independent scoring. Follow with a deterministic Mirage variant and a matched no-decoy control.

Done when: both authorization gates remain enforced; errors and inconclusive trials are recorded; decoys preserve behavior; any benefit is measured against a control.

Own attack_lab/ and protectors/mirage/. Coordinate LLVM hooks with A. Branch: feat/lab-mirage. Issue #10. Technical Documentation 10B (two pages).

Four roles continued

C Frontend & Product Experience

Build the complete developer workflow: supported-input guidance, upload, protection settings, job progress, errors, artifacts and evidence. Connect to the real API and create a reproducible demonstration. Own platform/dashboard/ and presentation assets.

First delivery: a usable interface that submits a real LLVM job and displays its actual status and report, followed by an ImageLab walkthrough. Coordinate job-listing and download endpoints with A; those capabilities are not in the documented API baseline.

Done when: identifiers, hashes and results come from actual jobs; loading, empty, unauthorized and failed states work; the demo can be repeated without special local knowledge.

Branch: feat/frontend. Issue #11. Technical Documentation 10C.

D Signature, Sentinel & Verification

Make build records verifiable. Start with an offline signed manifest linked to the final artifact digest, then add machine-readable policy records. Watermarking is a separate optional research task. Own provenance/ and sentinel/.

First delivery: manifest creation, public-key verification, clear failure states and tests for modified artifacts, modified manifests, wrong keys and malformed data. Keep private keys out of the repository and logs.

Done when: valid records verify against an explicitly trusted key and invalid records fail. Later policy and watermark work must have its own acceptance evidence; a policy notice alone cannot force compliance.

Branch: feat/signature-sentinel. Issue #12. Technical Documentation 10D.

How the work meets

A produces and integrates protected builds. B tests reconstruction and decoys. D signs records for fixed artifact bytes. C presents the real workflow and evidence. Shared contracts need review by the affected roles. External benchmark issue #7 spans all four roles.

Get started

1 Choose a bounded first task

Read your GitHub issue and Technical Documentation 10. Record your workstream, first deliverable, owned paths, and acceptance tests. Follow the current `AGENTS.md`, `ARCHITECTURE.md`, and module README. If their ownership descriptions conflict, ask the integration lead to reconcile them before editing shared files.

2 Check out your role branch

For a fresh clone, the example below checks out role A. Substitute your role branch from pages 3-4. For an existing checkout, inspect git status and follow CONTRIBUTING.md before switching.

```
git clone https://github.com/Youssef-Boussabah/AEGIS.git
cd AEGIS
git switch feat/core-shield
git merge --ff-only origin/dev/integration
git rev-parse HEAD
```

The documented development baseline is Linux x86-64, Python 3.11 or newer, and matching LLVM/Clang 22.x tools. The recorded runs used Python 3.14.7 and LLVM 22.1.8. See Technical Documentation 09 for the complete setup and Shield/ImageLab commands.

```
uv venv .venv
uv pip install --python .venv/bin/python \
  -c core/requirements-dev.lock \
  -e './core[dev]' -e './platform/api[dev]' \
  -e './platform/cli[dev]'
.venv/bin/pytest
```

That last command runs the three Python package suites. The real LLVM adapter suite needs its own paths and tool arguments; it is not included in the default root test selection.

3 Reproduce before changing

Run the tests relevant to your module and save the command and result. Build one small working change, then test the intended behavior and its failure cases. If a tool is missing, say the test was not run. Keep `.aegis/`, generated builds, tokens, and private keys outside commits.

4 Hand over something reviewable

Your pull request should explain what changed, which files changed, how to run it, which tests passed or failed, and what remains unresolved. Target `dev/integration`. Follow the repository's rules on when to commit, push, and open a PR; using an agent does not grant it extra permissions.

First-day outcome: a working environment, reproduced baseline tests, a scoped implementation plan, and agreement on any dependency that blocks your first deliverable.

Work with your coding agent

Use whichever coding agent you prefer. Give it both documents, the repository, and your issue. Fill in the assignment below. The full working brief in Technical Documentation 12 adds the detailed contracts and handoff format.

Copy this brief

You are helping implement AEGIS, a software protection platform intended for desktop software, web applications, mobile apps, games, server software, embedded systems, and authorized console builds. We use different backends for different execution environments. Our goal is to preserve intended behavior while increasing the measured difficulty of reverse engineering and reconstruction.

My role is [A-D and name]. My issue is [link]. My allowed paths are [paths]. The first deliverable is [specific behavior]. Its acceptance tests are [checks]. Repository actions I authorize are [edit and test only, or named additional actions].
Read AEGIS_USER_GUIDE, the relevant sections of AEGIS_TECHNICAL_DOCUMENTATION, AGENTS.md, ARCHITECTURE.md, and the module README. Inspect the current branch, commit, worktree, and code before relying on a document's status claim. Explain any mismatch.

The documented baseline contains Core, API, CLI, a textual LLVM 22 adapter, a prototype arithmetic transformation, and ImageLab. The adapter returns IR and a report; compiling and testing the application is a separate step. Shield v2, Mirage, Signature, Sentinel, the real Lab, optimizer, dashboard, and other backends are planned unless current code and tests establish otherwise.

Start by summarizing the relevant implementation, proposing a small change, identifying affected interfaces, and naming the tests you will run. Then implement within my assigned scope. Keep shared models, adapter protocols, and schemas compatible; route necessary changes through their owners.

Preserve program behavior, artifact hashing, path checks, subprocess limits, and both Lab authorization checks. Use independent expected results where possible. Inspect final compiled artifacts when evaluating transformations. Keep development tests separate from hidden evaluation data.

ImageLab, PhotoCraft, reVC, and Mattermost are test targets. Do not add application-specific protection rules or tune against hidden answers. Record tools, model identity, settings, budgets, hashes, and failed trials. Do not invent measurements or treat successful transformation as proof of security.

Finish with the files changed, exact test commands and outcomes, remaining limitations, contract questions, and the next concrete step. Write a short handoff note tied to the commit so another engineer or agent can continue.

Keep the shared understanding current

After an accepted change, update the affected module documentation and record important decisions. Handoffs should live with the repository. A conversation is useful working context; it is not the only record of what the team has built.

Reference index: Technical Documentation 13. Both guides use roles A-D; technical evidence remains the historical c995857 baseline.